MedQuick Project

Software Design Document

Digital Assignment 2 / Review 2

24BRS1009
24BRS1347

Medicine & Emergency Essentials Delivery Platform

Technology Stack:
MongoDB · Express.js · React.js · Node.js

Architecture:
3-Tier Modular Monolith

GitHub Repository:
https://github.com/SuAsPra/MedQuick_MERN_project

Figma Prototype:
https://www.figma.com/design/cVr0YrCWQ9vzvpH7YOpG79/MED_Quick-Pages?node-id=0-1&t=K7QtAcwjEVOsuzCD-1

1. Project Overview

MedQuick is a healthcare-focused ecommerce platform designed to enable users to browse and order medicines, first-aid products, emergency supplies, health devices, and other daily healthcare essentials through a centralized web application.

The project is developed using the MERN stack, consisting of a React frontend, Express.js and Node.js backend, and MongoDB database. The application follows a modular architecture where frontend functionality is organized into independent feature modules, while backend functionality is separated into routes, middleware, controllers, and Mongoose models.

The platform provides customer features such as authentication, product browsing, search, cart management, checkout, wishlist, reviews, addresses, and order management. Administrators are provided with centralized functionality for managing products, inventory, customers, and orders.

MedQuick extends a reusable MERN ecommerce foundation toward a healthcare-specific platform. The healthcare design includes **8 product categories**, **16 healthcare products**, medicine-specific product attributes, prescription-required product indicators, and a planned prescription verification dashboard.

Project Scope

Customer Features

- User registration and authentication
- Browse healthcare products
- Keyword-based product search
- Product details and reviews
- Shopping cart management
- Wishlist management
- Checkout and address management
- Order placement and tracking

Administrative Features

- Product management
- Inventory management
- Order management
- Customer management
- Healthcare product information management
- Prescription verification module *(planned)*

Project Objectives

The objectives of MedQuick are:

1. To provide a centralized platform for ordering medicines and emergency essentials.
2. To create a modular and maintainable MERN application.
3. To separate customer and administrator responsibilities through role-based functionality.
4. To support healthcare-specific product information and future prescription workflows.
5. To provide a scalable foundation for future deployment and feature expansion.

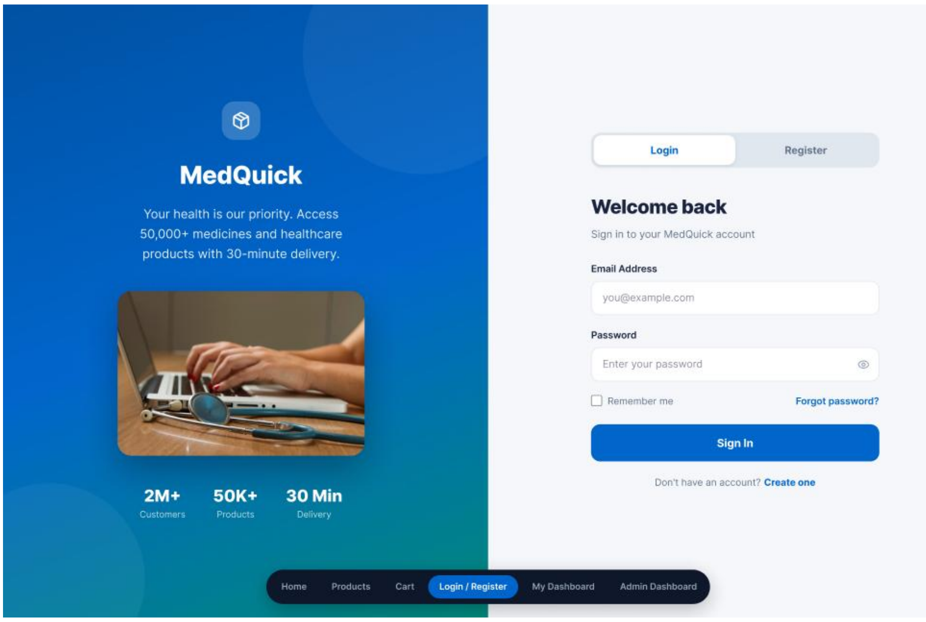


Figure 1: MedQuick Healthcare Delivery Platform – Customer Login Interface

2. Requirements to Design Mapping

The software design of MedQuick was derived from the requirements identified during Digital Assignment 1. User stories, functional requirements, MoSCoW prioritization, and UI wireframes were used as the foundation for designing the application modules.

The requirements were mapped to specific frontend and backend modules to maintain traceability between what users need and how the system is designed. This approach improves maintainability and ensures that major user requirements are represented in the implementation.

2.1 Requirement Traceability

User Requirement	MedQuick Design Module
User registration and login	Authentication Module
Browse healthcare products	Products Module
Search products	Product Listing and Search
View medicine/product details	Product Details Component
Add products to cart	Cart Module
Manage wishlist	Wishlist Module
Manage addresses	Address Module
Checkout and place orders	Checkout and Order Modules
View order history	Order Module
Write product reviews	Review Module
Manage user profile	User Module
Manage products	Admin Product Module
Manage orders	Admin Order Module
Manage categories and brands	Categories and Brands Modules
Identify prescription-required products	Healthcare Product Attributes
Verify prescriptions	Admin Prescription Module (Planned)

The traceability between requirements and modules allows MedQuick to support future changes without redesigning the complete system. For example, a new healthcare feature can be added as an independent module or by extending an existing feature module.

2.2 User Stories as the Basis for Design

MedQuick uses user stories to represent system requirements from the perspective of customers and administrators.

The user stories were created and managed using **GitHub Issues**, with each issue representing a specific requirement or feature. The issues were then organized using a **GitHub Projects Kanban Board** to track their development status.

Examples of user requirements include:

- Users should be able to register and securely log in.
- Users should be able to browse medicines and healthcare products.
- Users should be able to add products to a shopping cart.
- Users should be able to place and track orders.
- Administrators should be able to manage products and inventory.
- Administrators should be able to update order status.
- The system should identify products requiring prescriptions.

These user stories directly influenced the feature-based modular structure of the frontend and the route-controller-model organization of the backend.

GitHub User Stories

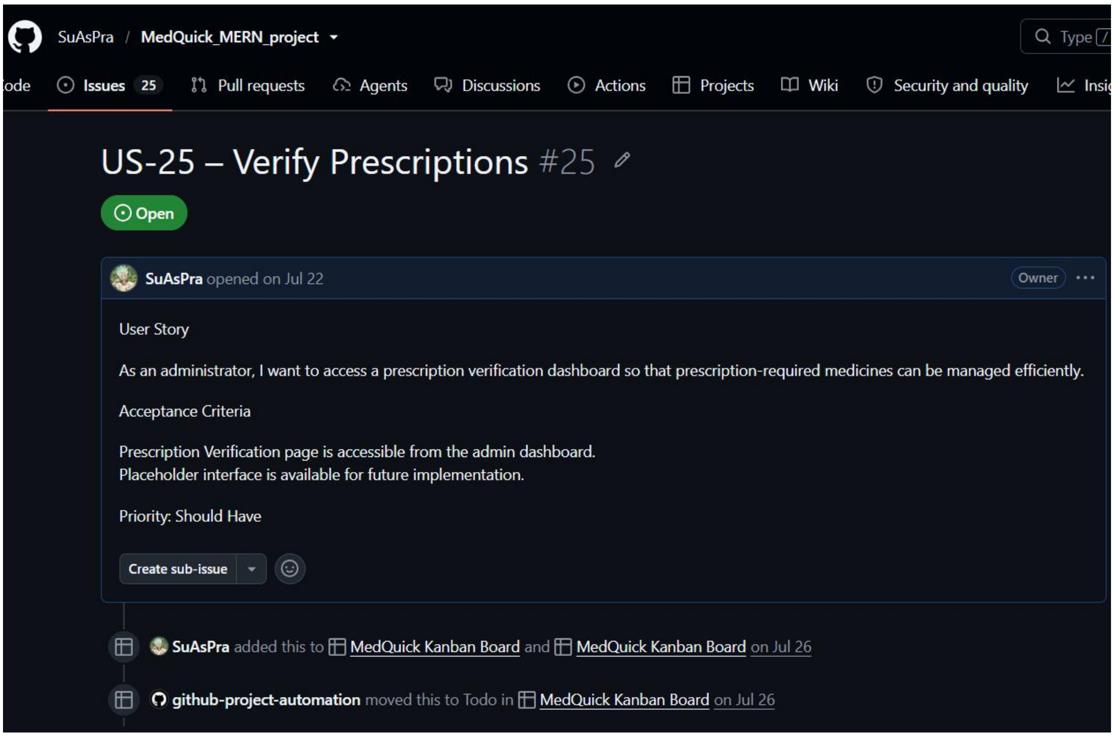


Figure 2: MedQuick User Stories Managed Using GitHub Issues

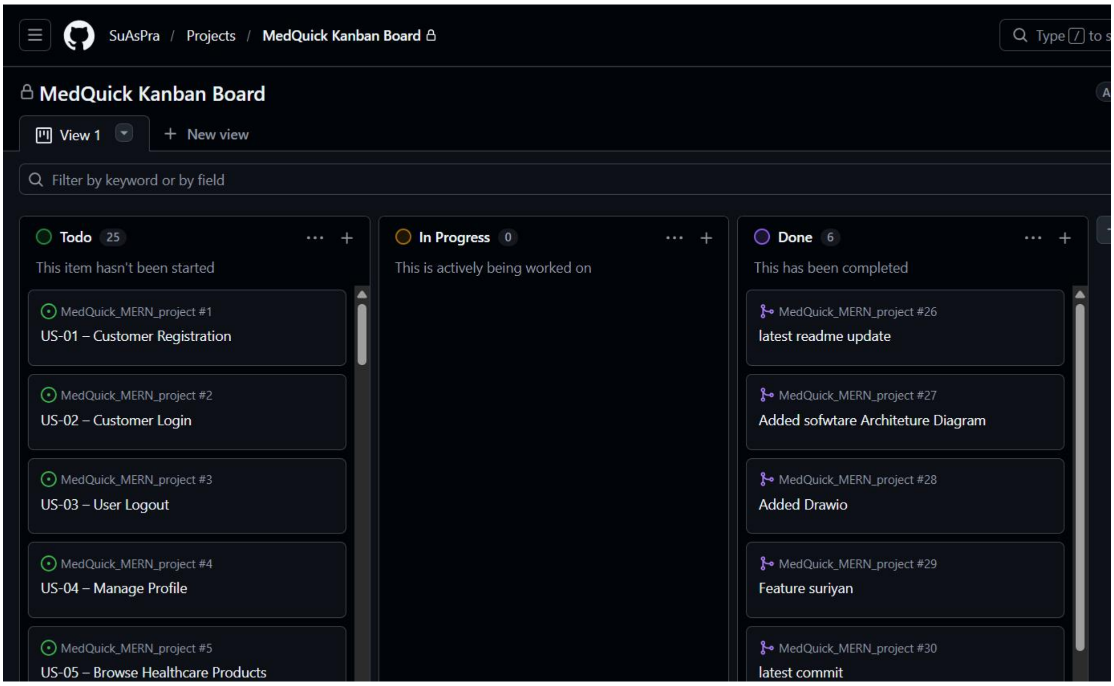


Figure 3: **MedQuick User Story Progress Managed Using GitHub Projects**

If both screenshots make Page 2 too crowded, move the Kanban Board screenshot to the bottom of Page 8 or omit it from the PDF. Since the PDF is limited to 10 pages, **GitHub Issues is more important for this page**.

3. Design Principles Applied

The MedQuick system was designed using modularity, abstraction, high cohesion, and low coupling. These principles help organize the application into manageable components and reduce the impact of future changes.

The frontend follows a **feature-based modular structure**, while the backend separates responsibilities using routes, middleware, controllers, and Mongoose models.

3.1 Modularity

Modularity refers to dividing a software system into independent and manageable modules, where each module focuses on a specific functionality. MedQuick demonstrates strong modularity through its feature-based frontend structure.

Frontend Feature Structure

```
front/src/features/
├── address
├── admin
├── auth
├── brands
├── cart
├── categories
├── checkout
```

```
├── order
├── products
├── review
├── user
└── wishlist
```

Each feature contains functionality related to a specific business domain. This structure separates product API communication, state management, and user interface components into organized modules.

As a result, changes to one feature have limited impact on unrelated features. For example, modifications to the Cart module do not normally require changes to the Authentication or Wishlist modules.

Benefits of Modularity

- Easier maintenance and debugging
- Clear separation of features
- Independent development of modules
- Better organization of source code
- Easier testing
- Easier addition of future features

The modular structure is particularly useful for MedQuick because future healthcare functionality, such as prescription upload or delivery tracking, can be added as separate modules without redesigning the entire application.

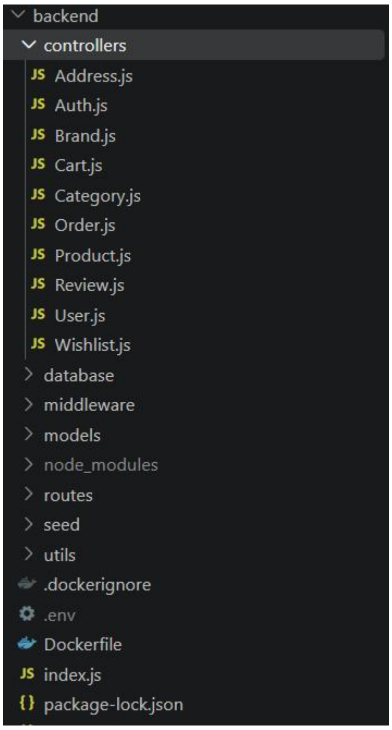


Figure 4: Feature-Based Modular Structure of the MedQuick Backend

3.2 Abstraction

Abstraction is used to hide unnecessary implementation details and provide simplified operations to other parts of the system.

In MedQuick, abstraction is demonstrated through feature-specific API modules.

For example, the Product API module provides operations such as:

```
fetchProducts(filters)
fetchProductById(id)
addProduct(data)
updateProductById(update)
deleteProductById(id)
```

The user interface does not need to know how HTTP requests are created or how Axios communicates with the backend.

Instead, API communication is abstracted into a dedicated layer.

Product Data Flow

React Components
↓
Redux / Product Logic
↓
ProductApi.jsx
↓
Axios
↓
Express REST API

This means that React components can focus on displaying information and handling user interaction while the API module handles communication with the backend.

Address.js ProductApi.jsx

front > src > features > products > ProductApi.jsx > addProduct

```
export const addProduct=async(data)=>{
}
export const fetchProducts=async(filters)=>{

    let queryString=''

    if(filters.brand){
        filters.brand.map((brand)=>{
            queryString+=`brand=${brand}&`
        })
    }
    if(filters.category){
        filters.category.map((category)=>{
            queryString+=`category=${category}&`
        })
    }
    if(filters.pagination){
        queryString+=`page=${filters.pagination.page}&limit=${filters.pagination.limit}&`
    }
    if(filters.sort){
        queryString+=`sort=${filters.sort.sort}&order=${filters.sort.order}&`
    }
    if(filters.user){
        queryString+=`user=${filters.user}&`
    }
    if(filters.search && filters.search.trim() !== ''){
        queryString+=`search=${encodeURIComponent(filters.search.trim())}&`
    }

    try {
        const res=await axiosi.get(`/products?${queryString}`)
        const totalResults=await res.headers.get("X-Total-Count")
        return {data:res.data,totalResults:totalResults}
    } catch (error) {
        throw error.response.data
    }
}
```

Source:
front/src/features/products/ProductApi.jsx

For your PDF, **copy your actual** fetchProducts() **function** from this file.

Use approximately **8–15 lines**, not the entire file.

Then place it like this:

// ProductApi.jsx
// Paste your actual fetchProducts()// function here from the project

Important: When making the final PDF, replace this placeholder with your actual code. Do not use invented code.

Why This Demonstrates Abstraction

The API implementation details are hidden inside ProductApi.jsx. Other parts of the application interact with product operations through reusable functions rather than directly creating HTTP requests.

This abstraction provides the following benefits:

- UI components remain simpler.
- API logic is centralized.
- HTTP implementation can be modified independently.
- Code duplication is reduced.
- Future backend changes are easier to manage.

3.3 High Cohesion

Cohesion refers to how closely related the responsibilities within a module are. A highly cohesive module focuses on a specific purpose instead of handling many unrelated tasks.

MedQuick demonstrates high cohesion by organizing state management and functionality according to individual business features.

For example, the product state management is grouped within:

```
front/src/features/products/ProductSlice.jsx
```

The Product Slice contains product-related functionality such as:

- Product state
- Initial state
- Asynchronous operations
- Reducers
- Extra reducers
- Actions
- Selectors

Similarly, MedQuick contains separate state modules for other business features:

```
ProductSlice
OrderSlice
CartSlice
WishlistSlice
ReviewSlice
AuthSlice
```

Each module focuses primarily on its own business domain. This improves code organization and makes the application easier to maintain.

C

```
ProductSlice.jsx ×
front > src > features > products > ProductSlice.jsx > ...
const initialState={
    products:[],
    totalResults:0,
    isFilterOpen:false,
    selectedProduct:null,
    errors:null,
    successMessage:null
}

export const addProductAsync=createAsyncThunk("products/addProductAsync",async(data)
    const addedProduct=await addProduct(data)
    return addedProduct
})
export const fetchProductsAsync=createAsyncThunk("products/fetchProductsAsync",async
    const products=await fetchProducts(filters)
    return products
})
export const fetchProductByIdAsync=createAsyncThunk("products/fetchProductByIdAsync"
    const selectedProduct=await fetchProductById(id)
    return selectedProduct
})
export const updateProductByIdAsync=createAsyncThunk("products/updateProductByIdAsyn
    const updatedProduct=await updateProductById(update)
    return updatedProduct
})
export const undeleteProductByIdAsync=createAsyncThunk("products/undeleteProductById
    const unDeletedProduct=await undeleteProductById(id)
    return unDeletedProduct
})
export const deleteProductByIdAsync=createAsyncThunk("products/deleteProductByIdAsyn
    const deletedProduct=await deleteProductById(id)
    return deletedProduct
})

const productSlice=createSlice({
    name:"productSlice",
    initialState:initialState,
    reducers:{
```

front/src/features/products/ProductSlice.jsx

For the PDF, insert a **small actual section** of your ProductSlice.jsx containing relevant parts such as:

- initialState
- createAsyncThunk
- createSlice
- reducers

Do not paste the complete file.

Suggested size:

12–20 lines maximum

Why This Demonstrates High Cohesion

All the responsibilities inside the Product Slice are directly related to managing product data and product state.

For example:

```
ProductSlice
  ├── Fetch Products
  ├── Store Products
  ├── Loading State
  ├── Error State
  └── Product Actions
```

Unrelated functionality such as authentication, orders, or wishlist management is handled by separate modules.

This demonstrates a high level of cohesion.

3.4 Low Coupling

Low coupling means that individual modules should have minimal dependency on the internal implementation of other modules.

MedQuick separates its application into multiple layers:

```
React UI Components
      ↓
Redux Toolkit
      ↓
Feature API Module
      ↓
Centralized Axios Client
      ↓
Express REST API
      ↓
Routes / Middleware
      ↓
Controllers
      ↓
Mongoose Models
      ↓
MongoDB
```

Each layer has a specific responsibility and communicates with adjacent layers instead of directly accessing distant layers.

For example, a customer adding a product to the cart follows a flow similar to:

```
ProductCard
   ↓
Cart Action
   ↓
CartSlice
   ↓
CartApi
   ↓
Axios
```

↓
Cart REST Endpoint

The Product UI component does not directly communicate with the database or MongoDB models.

This separation reduces dependencies and makes changes easier to manage.

Code Snippet 3 — Centralized Axios Configuration

Source:

front/src/config/axios.js

Insert this code:

```
import axios from 'axios';
export const axiosi = axios.create({
  withCredentials: true,
  baseURL: process.env.REACT_APP_BASE_URL
});
```

Why This Demonstrates Low Coupling

Instead of configuring Axios separately in every React component, MedQuick uses a centralized API client.

This means that configuration such as the backend URL can be changed in one location.

Benefits include:

- Reduced code duplication
- Easier configuration changes
- Consistent API communication
- Reduced dependency between UI and HTTP implementation

3.5 Separation of Responsibility

MedQuick also separates authentication logic from business logic through middleware.

JWT verification is handled through:

back/middleware/VerifyToken.js

Instead of repeating authentication checks inside every controller, the middleware is responsible for verifying user authentication before protected operations are performed.

VerifyToken.js

backend > middleware > VerifyToken.js > ...

```javascript
exports.verifyToken=async(req,res,next)=>{
    try {
        // extract the token from request cookies
        const {token}=req.cookies

        // if token is not there, return 401 response
        if(!token){
            return res.status(401).json({message:"Token missing, please login again"})
        }

        // verifies the token
        const decodedInfo=jwt.verify(token,process.env.SECRET_KEY)

        // checks if decoded info contains legit details, then set that info in req.user a
        if(decodedInfo && decodedInfo._id && decodedInfo.email){
            req.user=decodedInfo
            next()
        }

        // if token is invalid then sends the response accordingly
        else{
            return res.status(401).json({message:"Invalid Token, please login again"})
        }

    } catch (error) {

        console.log(error);

        if (error instanceof jwt.TokenExpiredError) {
            return res.status(401).json({ message: "Token expired, please login again" });
        }
        else if (error instanceof jwt.JsonWebTokenError) {
            return res.status(401).json({ message: "Invalid Token, please login again" });
        }
        else {
            return res.status(500).json({ message: "Internal Server Error" });
```

Source:

back/middleware/VerifyToken.js

Insert the JWT verification portion of your **actual code** here.

Use approximately **8–15 lines**.

Why This Demonstrates Separation of Responsibility

The middleware is responsible for authentication verification, while controllers focus on business operations.

For example:

VerifyToken Middleware
↓
Authentication Verification

Controller
↓
Business Logic

Model
↓
Database Operations

This reduces code duplication and improves maintainability.

3.6 SOLID Principles Evaluation

The SOLID principles were evaluated based on the actual architecture and code structure of MedQuick.

Since MedQuick primarily uses React components, JavaScript modules, Redux Toolkit, and a layered MERN architecture, not every traditional object-oriented SOLID principle is equally applicable.

The project therefore does **not claim complete implementation of all five principles**.

SOLID Principle	MedQuick Evidence	Assessment
S — Single Responsibility	Feature-specific slices, API modules, middleware and controllers	Strong
O — Open/Closed	Reusable components and extensible feature structure	Partial
L — Liskov Substitution	Limited inheritance/interface usage	Not strongly applicable
I — Interface Segregation	Separate feature-specific API modules	Partial
D — Dependency Inversion	Layer separation and API abstraction	Partial

---

S — Single Responsibility Principle

The Single Responsibility Principle states that a module should have one primary responsibility.

Examples from MedQuick include:

ProductApi.jsx
→ Product API communication

ProductSlice.jsx
→ Product state management

VerifyToken.js
→ Authentication verification

ProductCard.jsx
→ Product UI presentation

Each module has a clearly defined responsibility.

Assessment: Strongly Demonstrated

O — Open/Closed Principle

MedQuick uses reusable components and modular feature structures that can be extended with new functionality.

For example, new healthcare product attributes or additional product components can be added without redesigning unrelated features.

Assessment: Partially Demonstrated

L — Liskov Substitution Principle

The Liskov Substitution Principle is generally associated with inheritance and substitutable object types.

The current MedQuick implementation does not heavily use classical inheritance or interface hierarchies.

Therefore, this principle is not strongly applicable to the current React and JavaScript architecture.

Assessment: Limited Applicability

I — Interface Segregation Principle

Feature-specific API modules provide operations related to individual business domains.

For example:

```
ProductApi
CartApi
OrderApi
WishlistApi
```

Modules use functionality relevant to their own feature rather than depending on one large interface.

Assessment: Partially Demonstrated

D — Dependency Inversion Principle

MedQuick separates high-level UI functionality from lower-level API and database implementation.

For example:

React Component
↓
Redux / API Module
↓
Axios
↓
REST API

The React interface does not directly depend on MongoDB implementation details.

Assessment: Partially Demonstrated

4. High-Level Architecture

MedQuick follows a **3-Tier Client-Server Architecture implemented as a Modular Monolith**.

The application is organized into three main logical tiers:

1. **Presentation Tier** — React user interface
2. **Application Tier** — Node.js and Express.js backend
3. **Data Tier** — MongoDB database

Within these tiers, the application is further organized into independent modules and layers to improve maintainability and separation of responsibilities.

The complete system is packaged for local development using **Docker and Docker Compose**, allowing the frontend, backend, and database services to run in a consistent environment.

4.1 Architecture Style

3-Tier Modular Monolith Architecture

MedQuick was designed as a **3-Tier Modular Monolith**.

The system is deployed as a single integrated application rather than being divided into independent microservices. However, the internal code is organized into modular features and layers.

Architecture Tiers

Presentation Tier

The frontend is built using:

- React.js
- Redux Toolkit
- Material UI
- Axios

This layer is responsible for:

- User interface
- Customer interaction
- Admin dashboard interface
- Application state management
- API communication

Application Tier

The backend is built using:

- Node.js
- Express.js
- JWT Authentication
- Middleware
- Controllers
- REST APIs

This layer handles:

- Business logic
- Authentication
- Authorization
- Product operations
- Cart operations
- Order processing
- Administrative operations

Data Tier

The database layer uses:

- MongoDB
- Mongoose ODM

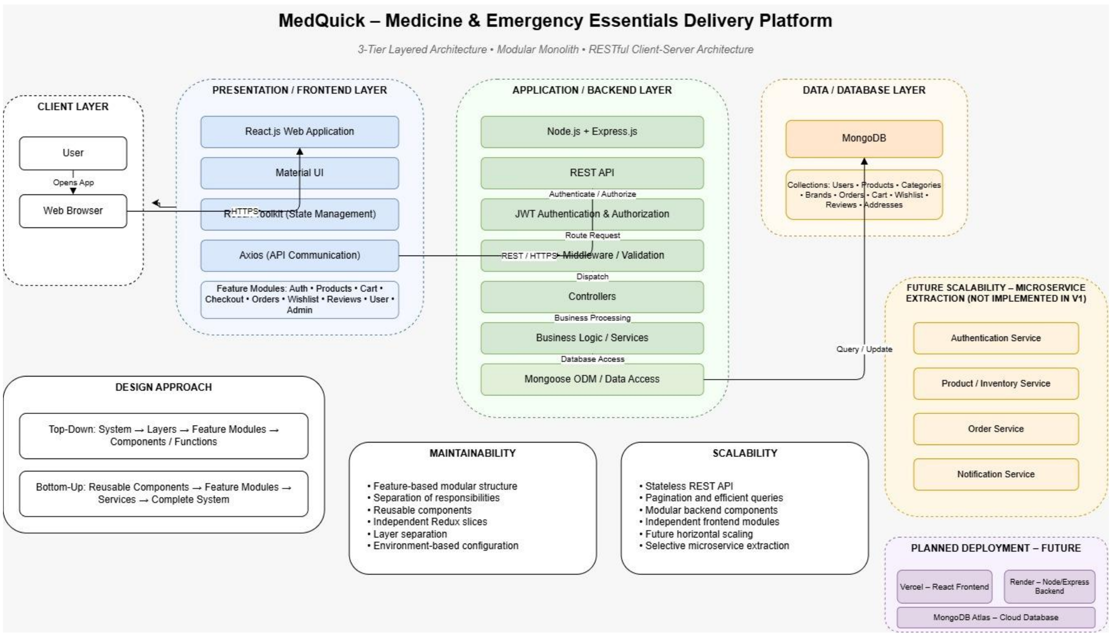
This layer stores:

- Users
- Products
- Categories
- Brands
- Orders
- Reviews
- Cart information
- Addresses
- Wishlist information

4.2 System Data Flow

The main data flow in MedQuick is:

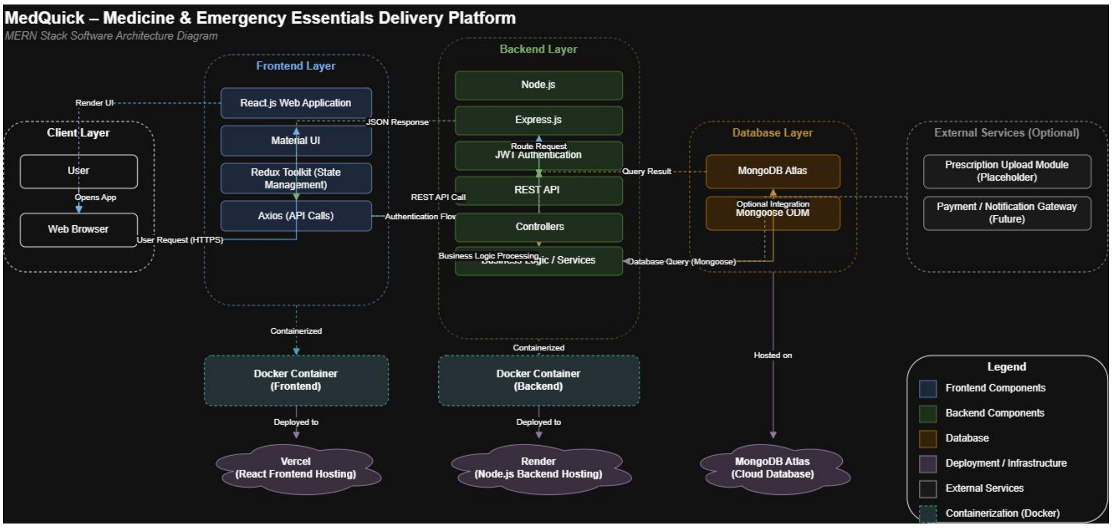
React User Interface
↓
Redux Toolkit
↓
Feature API Modules
↓
Centralized Axios Client
↓
Express REST API
↓
Routes / Middleware
↓
Controllers
↓
Mongoose Models
↓
MongoDB



The customer and administrator interfaces communicate with the backend through REST APIs. Authentication and authorization are handled through middleware before protected operations reach the appropriate controllers.

Controllers process business operations and communicate with Mongoose models to store or retrieve data from MongoDB.

This layered flow ensures that user interface components do not directly communicate with the database.



4.3 Why This Architecture Was Chosen

The 3-Tier Modular Monolith architecture was selected because it is suitable for the current size and scope of MedQuick.

Reason 1 — Simplicity

The project is developed by a small team of two student developers. A modular monolith is easier to develop, test, debug, and deploy compared with a distributed microservices architecture.

Reason 2 — Maintainability

The frontend is organized into feature modules such as:

Auth
Products
Cart
Orders
Wishlist
Reviews
Address
Admin

The backend separates responsibilities into:

Routes
Middleware
Controllers
Models

This makes individual features easier to modify without affecting the complete application.

Reason 3 — Clear Separation

The three-tier structure separates:

User Interface
↓

Application Logic
↓
Data Storage

This reduces direct dependencies between the frontend and database.

Reason 4 — Future Scalability

Although MedQuick currently uses a modular monolith architecture, the modular structure provides a foundation for future improvements.

For example, future features such as:

- Prescription verification
- Delivery tracking
- Payment integration
- Notification services
- OCR prescription processing
- Mobile application integration

can be developed as separate modules or potentially extracted into independent services if the system grows significantly.

4.4 Containerization and Deployment

For local development, MedQuick uses:

- Docker
- Dockerfiles
- Docker Compose

Containerization provides a consistent environment for running the application and its dependencies.

The Docker environment can include services such as:

Frontend Container
↓
Backend Container
↓
MongoDB Container

This reduces differences between development environments and makes the project easier to reproduce on another machine.

The architecture also provides a foundation for future cloud deployment.

Possible future deployment options include:

- Frontend deployment service
- Backend cloud server
- Managed MongoDB database

- Container registry
- CI/CD pipeline

These deployment services are considered future extensions and are not claimed as part of the current implementation unless they have actually been configured in your project.

5. User Interface Design

The MedQuick user interface was designed to provide a simple and familiar healthcare shopping experience. The design was initially created using Figma and was based on the functional requirements and user stories developed during Digital Assignment 1.

The interface separates customer-facing functionality from administrative functionality while maintaining consistent navigation, layout, colors, typography, and interaction patterns.

The design focuses on allowing users to quickly find healthcare products and complete common tasks such as browsing, searching, adding products to the cart, and placing orders.

5.1 User-Friendly Design Principles

The following principles were considered while designing the MedQuick interface:

1. Simple Navigation

The navigation structure provides access to major features such as:

- Home
- Products
- Categories
- Cart
- Wishlist
- Orders
- User Profile

This reduces the number of steps required for users to access frequently used functionality.

---

2. Clear Product Information

Product cards and product detail pages are designed to clearly display important information.

For healthcare-related products, the design can display:

- Product name
- Price
- Category
- Manufacturer
- Dosage
- Medicine type

- Expiry information
- Prescription requirement

This allows users to identify relevant information before adding a product to their cart.

---

3. Consistent Components

Reusable UI components are used throughout the application.

Examples include:

- Navigation bar
- Product cards
- Buttons
- Forms
- Alerts
- Dialog boxes

Material UI helps maintain consistency in spacing, typography, form elements, and responsive behavior.

---

4. Clear User Feedback

The interface provides feedback for user actions such as:

- Adding products to the cart
- Updating quantities
- Logging in
- Placing orders
- Updating profile information

This helps users understand whether an action was successfully completed.

SCREEN 1 — HOME PAGE

Insert your **MedQuick Figma Home Page**.

The screen should ideally show:

- MedQuick branding
- Navigation bar
- Healthcare banner
- Product categories
- Featured products
- Search or browsing entry point

Caption:

Figure 6: MedQuick Home Page Design

Design Explanation

The Home Page provides users with a clear starting point for exploring the platform. Healthcare categories and featured products help users quickly discover relevant products without navigating through multiple pages.

The visual hierarchy uses prominent banners and organized product sections to guide users from general browsing toward specific products.

SCREEN 2 — PRODUCT LISTING / BROWSING

Insert your **Product Listing Figma screen**.

It should ideally show:

- Product cards
- Product images
- Product names
- Prices
- Categories
- Search or filter controls
- Prescription badge where applicable

Caption:

Figure 7: Healthcare Product Listing Interface

Design Explanation

The product listing interface is designed to support efficient product discovery. Products are displayed using reusable cards to provide a consistent visual structure.

Search and category filtering reduce the effort required to find specific healthcare products. Prescription-required products are visually identified to provide users with important information before proceeding to purchase.

SCREEN 3 — PRODUCT DETAILS

Insert your **Product Details Figma screen**.

It should show:

- Product image
- Product name
- Price

- Description
- Manufacturer
- Dosage
- Medicine type
- Expiry date
- Prescription requirement
- Add to Cart button

Caption:

Figure 8: Healthcare Product Details Interface

Design Explanation

The Product Details screen provides detailed information before a user makes a purchase decision.

Healthcare-specific information is separated from general product information to improve readability. Important alerts, such as prescription requirements, are displayed prominently to reduce the possibility of users missing critical information.

5.2 Customer and Administrative Interface Design

MedQuick has two primary types of users:

- **Customers / Patients**
- **Administrators**

Although both users access the same MERN application, they are provided with different functionality based on their role.

The customer interface focuses on browsing and ordering healthcare products, while the administrator interface focuses on managing products, orders, customers, and healthcare-related inventory information.

This role-based separation improves usability because users are only presented with functionality relevant to their tasks.

SCREEN 4 — CART / CHECKOUT

Insert your **Cart or Checkout Figma screen**.

It should ideally show:

- Selected products
- Product quantity
- Price
- Subtotal
- Delivery address
- Checkout button

- Order summary

Caption:

Figure 9: MedQuick Cart and Checkout Interface

Design Explanation

The Cart and Checkout interface organizes order information in a clear sequence. Users can review selected products, update quantities, view the total cost, and continue toward order placement.

Keeping the order summary visible helps users understand their purchase before completing the checkout process. The interface reduces unnecessary steps by grouping related checkout information together.

SCREEN 5 — LOGIN / AUTHENTICATION

Insert your **Login or Signup Figma screen**.

It should show:

- MedQuick branding
- Email input
- Password input
- Login button
- Signup option
- Password recovery option, if present

Caption:

Figure 10: MedQuick User Authentication Interface

Design Explanation

The authentication interface uses a simple form layout with clearly labeled fields and limited distractions.

Authentication-related functionality is separated from shopping and administrative features, allowing users to focus on completing login or registration tasks. Clear validation and feedback messages improve usability when incorrect information is entered.

SCREEN 6 — ADMIN DASHBOARD

Insert your **Admin Dashboard Figma screen**.

It should ideally show:

- Dashboard navigation
- Product management
- Order management
- Customer management
- Inventory information
- Prescription verification placeholder
- Summary cards or statistics

Caption:

Figure 11: MedQuick Administrator Dashboard

Design Explanation

The administrator interface is designed around management tasks rather than product shopping.

Administrative functionality is grouped into clearly separated sections such as:

- Products
- Orders
- Customers
- Categories
- Inventory
- Prescription Verification

This improves usability by allowing administrators to quickly access management operations without navigating through customer-oriented pages.

5.3 Role-Based Interface Separation

The following table shows how the interface differs based on the user role.

Customer Interface	Administrator Interface
Browse products	Manage products
Search products	Update inventory
View product details	Manage orders
Add to cart	Update order status
Manage wishlist	Manage customers
Checkout	Manage categories/brands
View orders	Prescription verification placeholder
Manage profile and addresses	Administrative dashboard

The separation between customer and administrator interfaces also supports better software organization. Role-based access control ensures that administrative operations are not exposed as normal customer features.

5.4 Responsive and Consistent Design

MedQuick uses reusable interface components and Material UI to support a consistent visual design.

Common interface elements include:

- Buttons
- Forms
- Navigation components
- Product cards
- Dialog boxes
- Alerts
- Tables

Reusable components reduce unnecessary duplication and make future UI updates easier.

For example, a reusable product card can be used in:

Product Listing
↓
Wishlist
↓
Admin Product View

Instead of designing separate product card implementations for every page, the same component structure can be reused where appropriate.

This supports:

- Consistency
- Maintainability
- Faster development
- Easier UI updates

5.5 Healthcare-Specific UI Decisions

MedQuick adapts a standard ecommerce interface for healthcare product delivery by including healthcare-specific information.

The interface design supports the display of:

- Prescription requirement
- Manufacturer
- Dosage
- Medicine type
- Expiry date

A **Prescription Required** indicator is designed to be visually prominent so that users can identify products requiring additional verification.

This is an important design decision because healthcare products may require more information than ordinary ecommerce products.

6. Key Design Decisions

The following design decisions were made during the development of MedQuick to improve maintainability, usability, and future extensibility.

6.1 Feature-Based Frontend Architecture

Decision

The React frontend was organized using **feature-based modules** instead of grouping all components, APIs, and state files by technical type.

Example:

```
features/
 ├── auth/
 ├── products/
 ├── cart/
 ├── order/
 ├── wishlist/
 ├── address/
 ├── review/
 └── admin/
```

Why?

Each feature keeps its related components, API communication, and state management close together.

For example:

```
products/
 ├── ProductApi.jsx
 ├── ProductSlice.jsx
 └── components/
```

This makes the code easier to understand and maintain.

Benefit

- High modularity
- Better cohesion
- Easier maintenance
- Easier feature development
- Easier future expansion

6.2 Redux Toolkit for State Management

Decision

MedQuick uses **Redux Toolkit** for managing shared application state.

Feature-specific slices are used for areas such as:

- Products
- Cart
- Orders
- Authentication
- Wishlist
- Reviews

Why?

Ecommerce applications contain data that must be shared across multiple components.

For example:

User logs in
↓
Authentication state updated
↓
Navbar updates
↓
Protected pages become accessible

Similarly:

User adds product
↓
Cart state updated
↓
Cart count updates
↓
Checkout accesses cart data

Redux Toolkit provides a centralized and structured approach for handling this shared state.

Benefit

- Predictable state management
- Reduced prop drilling
- Better separation of state logic
- Easier debugging
- Reusable asynchronous operations

6.3 3-Tier Modular Monolith Architecture

Decision

MedQuick was implemented as a **3-Tier Modular Monolith**.

Presentation Layer

↓
Application Layer
↓
Data Layer

Why?

The project is currently developed by a small team of two student developers.

Using microservices would introduce additional complexity such as:

- Multiple deployments
- Inter-service communication
- Distributed debugging
- Additional infrastructure
- More complex testing

A modular monolith provides sufficient structure while remaining easier to develop and maintain.

Benefit

- Simpler development
- Easier debugging
- Lower deployment complexity
- Clear architecture
- Suitable for current project scope

6.4 Role-Based Customer and Admin Design

Decision

MedQuick provides separate functionality for:

- Customers
- Administrators

Customers interact with:

Products
Cart
Wishlist
Checkout
Orders
Profile
Addresses

Administrators interact with:

Products
Orders
Inventory
Customers
Categories
Brands
Prescription Verification

Why?

Customer and administrative users have different goals.

Customers need a simple shopping and ordering experience, while administrators need management tools.

Providing both sets of functionality on the same interface would create unnecessary complexity.

Benefit

- Better usability
- Improved security
- Clearer navigation
- Role-based access
- Separation of responsibilities

6.5 Healthcare-Specific Product Schema

Decision

The existing ecommerce product schema was extended with healthcare-specific attributes.

Examples include:

- requiresPrescription
- manufacturer
- expiryDate
- dosage
- medicineType

Why?

A standard ecommerce product structure is insufficient for representing healthcare and medicine-related products.

MedQuick requires additional information to better describe healthcare products.

For example:

```
Standard Product
   ↓
Name
Price
Description

MedQuick Product
   ↓
Name
Price
Description
Manufacturer
Dosage
Medicine Type
```

Expiry Date
Prescription Requirement

Benefit

- Healthcare-specific product information
- Improved product clarity
- Better future extensibility
- Supports prescription-related workflows
- Distinguishes MedQuick from a standard ecommerce application

---

6.6 Summary of Design Decisions

Use this table near the bottom of the page.

Design Decision	Main Reason	Main Benefit
Feature-Based Architecture	Organize related functionality	Maintainability
Redux Toolkit	Manage shared state	Predictability
Modular Monolith	Suitable for project size	Simplicity
Role-Based Design	Different user requirements	Better usability
Healthcare Product Schema	Support medicine information	Domain extensibility

6.7 Current Project Metrics

The following metrics are based on the current MedQuick implementation and project verification records.

Project Metric	Value
Estimated source size	≈ 9 KLOC
JS/JSX source files	128
Frontend feature modules	14+
Architecture	3-Tier Modular Monolith
Database	MongoDB + Mongoose
Healthcare categories	8
Seed healthcare products	16
Order lifecycle states	6

Product Metrics

Product Metric	Value
Authentication	JWT + Cookie-based
Search	Keyword-based
Soft deletion	Implemented
Order snapshot	Implemented

Product Metric	Value
Customer feature areas	8+
Admin capabilities	Products, Orders, Inventory, Customers, Prescription placeholder

Important Design Observation

The **order snapshot** design preserves relevant product and address information at the time an order is created. Therefore, future changes to product information do not automatically alter historical order information.

This improves the consistency of order records.

7. Maintainability and Future Changes

MedQuick was designed with maintainability in mind. The application separates major responsibilities into independent modules and layers so that changes can be made without requiring modifications throughout the complete system.

The frontend uses feature-based modules, while the backend separates routes, middleware, controllers, and models.

This organization supports easier maintenance, debugging, testing, and future feature development.

7.1 How the Design Supports Maintainability

Feature-Based Frontend

The frontend is divided into independent business features:

```
Auth
Products
Cart
Checkout
Orders
Wishlist
Reviews
Address
User
Admin
```

For example, changes to the Wishlist feature can generally be made within the Wishlist module without modifying the Product or Authentication modules.

This reduces the impact of changes and improves maintainability.

Layered Backend

The backend follows a separation of responsibilities:

Routes
↓
Middleware
↓
Controllers
↓
Mongoose Models
↓
MongoDB

Each layer performs a different responsibility.

Layer	Responsibility
Routes	Define API endpoints
Middleware	Authentication and request processing
Controllers	Business logic
Models	Database structure and operations
MongoDB	Persistent data storage

This structure makes it easier to identify where a change should be implemented.

For example:

- Authentication changes → Middleware
- Product logic changes → Product Controller
- Database field changes → Product Model
- API endpoint changes → Routes

7.2 Reusable Components and Future Changes

MedQuick uses reusable components and centralized modules where possible.

Examples include:

- Product cards
- Navigation components
- Axios API configuration
- Redux slices
- Feature-specific API modules
- Authentication middleware

This reduces duplication.

For example, the same Product Card design can be reused across:

Product Listing
↓
Wishlist
↓
Admin Product View

A future UI improvement to the Product Card can therefore benefit multiple sections of the application.

7.3 Future Improvements

The current MedQuick implementation provides a foundation for future development.

Potential future improvements include:

1. Real Prescription Upload

The current project includes prescription-related product information and an administrative prescription verification placeholder.

A future version could implement:

```
Customer
  ↓
Upload Prescription
  ↓
Backend Storage
  ↓
Admin Verification
  ↓
Approve / Reject
```

---

2. OCR Prescription Processing

AI or OCR technology could be integrated to extract medicine information from uploaded prescriptions.

Possible flow:

```
Prescription Image
      ↓
OCR / AI Processing
      ↓
Medicine Information Extraction
      ↓
Admin Verification
```

This could assist administrators but should still include human verification for healthcare-related decisions.

---

3. Real-Time Delivery Tracking

A future delivery system could allow users to track order progress.

Possible statuses include:

```
Pending
  ↓
Confirmed
  ↓
Packed
```

```
  ↓
Out for Delivery
  ↓
Delivered
```

The current order lifecycle already provides a foundation for this feature.

---

4. Payment Integration

A future version could integrate a secure payment gateway.

Possible additions:

- Online payments
- Payment verification
- Refund management
- Transaction history

---

5. Notifications

The system could send notifications for:

- Order confirmation
- Order status updates
- Prescription approval
- Delivery updates
- Low stock alerts

---

6. Mobile Application

Because the backend provides REST APIs, a future mobile application could communicate with the existing backend.

Possible architecture:

```
React Web App
      │
      ├──────────────┐
      │          │
      ▼          ▼
   REST API    Mobile App
      │
      ▼
   MongoDB
```

This would allow the existing backend architecture to support additional client applications.

7.4 Project Metrics

The following metrics describe the approximate current size and structure of the MedQuick implementation.

Project Metric	Current Value
Estimated source size	≈ 9 KLOC
JS/JSX source files	128
Frontend feature modules	14+
Architecture	3-Tier Modular Monolith
Database	MongoDB + Mongoose
Healthcare categories	8
Seed healthcare products	16
Order lifecycle states	6

These metrics are intended to provide an approximate view of the project size and functionality and are **not formal industry measurements**.

7.5 Product and Quality Metrics

Product Metrics

Metric	MedQuick Value
Healthcare categories	8
Healthcare products	16
Order lifecycle stages	6
Healthcare product attributes	5
Customer feature areas	Auth, Products, Cart, Checkout, Orders, Wishlist, Reviews, Address
Admin capabilities	Products, Orders, Inventory, Customers, Prescription placeholder
Authentication	JWT + Cookie-based
Search	Keyword-based
Soft deletion	Implemented
Order snapshot	Implemented

Quality Verification Metrics

Verification Area	Result
Backend verification	PASS
Database seed verification	PASS
Frontend build	PASS
Customer flow	PASS
Admin flow	PASS
Legacy terminology scan	PASS
Remaining detected bugs during audit	0

Note: These results represent implementation verification and audit results. They should not be interpreted as formal automated test coverage percentages.

7.6 Effort and Reuse Metric

An estimated effort analysis was also performed for the project.

Development Approach	Estimated Effort
From-scratch development	≈ 24 person-months
Development with reuse/adaptation	≈ 7.5 person-months

MedQuick benefits from adapting an existing MERN ecommerce architecture and then extending it for the healthcare domain.

This demonstrates the value of software reuse.

The reusable architecture provided existing functionality such as:

- Authentication
- Product management
- Cart
- Orders
- Wishlist
- Reviews
- Admin functionality

The project team then focused on adapting these components for MedQuick-specific requirements.

7.7 Scheduling and Team Metrics

A 10-week Work Breakdown Structure and project schedule was prepared using project planning tools.

However, a final scheduling accuracy percentage should only be calculated after comparing planned and actual completion dates.

The following structure can be used:

Task	Planned Duration	Actual Duration	Variance
Requirements	—	—	—
Design	—	—	—
Development	—	—	—
Integration	—	—	—
Testing	—	—	—

Task	Planned Duration	Actual Duration	Variance
Documentation	—	—	—

Similarly, team contribution metrics should be based on actual GitHub activity.

Team Metric	Developer 1	Developer 2
Commits	Actual	Actual
Issues Completed	Actual	Actual
Features Contributed	Actual	Actual
Documentation	Actual	Actual
Diagrams	Actual	Actual
Testing	Actual	Actual

This approach avoids presenting estimated values as actual project measurements.

8. Conclusion

MedQuick was designed as a modular full-stack healthcare and emergency essentials delivery platform using the MERN stack. The software design was developed by transforming the requirements, user stories, and wireframes prepared during Digital Assignment 1 into a structured and maintainable software solution.

The project follows a **3-Tier Modular Monolith Architecture**, separating the presentation layer, application logic, and data layer. The frontend uses React, Redux Toolkit, Material UI, and Axios, while the backend uses Node.js, Express.js, middleware, controllers, Mongoose models, and MongoDB.

The design emphasizes important software engineering principles such as **modularity, abstraction, high cohesion, low coupling, separation of responsibilities, and maintainability**.

8.1 Summary of Design Approach

The main design approach used in MedQuick can be summarized as follows:

```
Requirements & User Stories
        ↓
UI Wireframes
        ↓
Feature-Based Design
        ↓
Frontend Modules
        ↓
REST API Backend
        ↓
MongoDB Data Layer
```

The user stories were mapped to independent application modules such as:

- Authentication
- Products

- Cart
- Checkout
- Orders
- Wishlist
- Reviews
- Address Management
- User Management
- Administration

This approach helps ensure that the system design remains connected to the original requirements.

8.2 Key Software Engineering Principles Applied

Principle	Application in MedQuick
Modularity	Feature-based frontend modules
Abstraction	API modules and centralized Axios client
High Cohesion	Feature-specific Redux slices
Low Coupling	Separation between UI, API, backend, and database
Single Responsibility	Separate components, middleware, controllers, and models
Maintainability	Modular and layered application structure
Reusability	Reusable UI components and shared API configuration

The project does not attempt to force every SOLID principle into the implementation. Instead, the principles were evaluated based on their actual relevance to the React and MERN architecture.

8.3 Main Design Choices

The most important design decisions were:

1. Feature-Based Frontend

Related functionality is grouped into independent features, improving modularity and maintainability.

2. Redux Toolkit State Management

Shared application state such as authentication, products, cart, and orders is managed in a structured and predictable manner.

3. 3-Tier Modular Monolith Architecture

The architecture provides sufficient separation and scalability while remaining suitable for a two-member college development team.

4. Role-Based Customer and Admin Interfaces

Customers and administrators receive different functionality based on their responsibilities.

5. Healthcare-Specific Product Information

The product structure was extended to support healthcare attributes such as manufacturer, dosage, expiry date, medicine type, and prescription requirements.

8.4 Maintainability and Future Growth

The modular design allows MedQuick to support future changes without requiring a complete redesign of the application.

Potential future improvements include:

- Real prescription upload and verification
- OCR-assisted prescription processing
- Online payment integration
- Real-time delivery tracking
- Order and delivery notifications
- Inventory alerts
- Mobile application integration
- Cloud deployment
- CI/CD automation

The existing REST API and modular structure provide a foundation for these future improvements.

8.5 Final Outcome

MedQuick demonstrates how an existing ecommerce architecture can be systematically adapted into a domain-specific healthcare delivery platform.

The final software design provides:

- A modular frontend structure
- Layered backend organization
- Clear separation of responsibilities
- Role-based functionality
- Healthcare-specific product support
- Docker-based local development
- Maintainable and reusable code organization

The design decisions were selected to balance **simplicity, maintainability, usability, and future extensibility**, making the system appropriate for the current scope of a college Software Engineering project while providing a foundation for future development.

Project Resources

At the bottom of the final page, add:

GitHub Repository

MedQuick GitHub Repository:
[Paste your GitHub Repository Link Here]

Figma Prototype

MedQuick Figma Design:
[Paste your Figma Prototype Link Here]

Architecture and Design Files

Editable Draw.io diagrams and exported images are available in the project's:

/design/

folder in the GitHub repository.